TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Self-Driving Car · Full Curriculum

3-session series (2 hrs each) · 11–15 yrs · Intermediate → Advanced

Full facilitator curriculum **PREMIUM**

Overview

Build a chassis-based car that navigates on its own, finishing with a race day.

DETAIL	
Track	Arduino Robotics & Electronics
Format	3-session series (2 hrs each)
Ages	11–15 yrs
Level	Intermediate → Advanced
Price	from KES 8,000
Standalone	No
Prerequisites	Servo motor control (free)

Course Overview

Self-Driving Car is a three-session build-and-code programme in which every student (or pair, for larger cohorts) assembles a real four-wheeled robot car from a bare chassis kit and, session by session, teaches it to drive itself. Students start with the mechanics and motor control that make the car move on command, then add sensing and decision-making so the car can follow a track and dodge obstacles without a human touching the controls.

The arc runs from manual control to full autonomy: Session 1 is "can it move?" (chassis assembly, L298N motor driver, directional code). Session 2 is "can it see the road?" (IR line-sensing and steering logic). Session 3 is "can it think for itself?" (ultrasonic distance sensing, obstacle-avoidance logic, and a combined line-plus-obstacle mode) — capped by a live Race Day where every car runs the same track and obstacle course for time and style points.

By the end, each student takes home a fully wired, fully programmed self-driving car chassis (Arduino Uno, L298N driver, IR line sensors, HC-SR04 ultrasonic sensor, all on a 4WD chassis) plus the complete code library from all three builds, so they can keep tuning and racing it at home.

Learning Outcomes

- Assemble a geared DC-motor chassis and correctly wire a dual H-bridge motor driver (L298N) for four-quadrant control.
- Write and debug Arduino functions for forward, reverse, pivot-turn and stop using PWM speed control.
- Explain how infrared reflectance sensors distinguish a black line from a white surface, and calibrate a sensor for reliable digital readings.
- Implement a closed-loop line-following algorithm that reacts to live sensor input in real time.
- Explain how the HC-SR04 measures distance using the speed of sound, and code a pulse-timing distance function from scratch.
- Combine two independent sensing systems (line + ultrasonic) into one decision-making loop — a first taste of sensor fusion.
- Test, iterate and tune a physical robot against a live performance target (Race Day), including systematic hardware/software debugging.

Materials Master List

ITEM	SPEC	QTY PER STUDENT/ TEAM	NOTES
Arduino Uno R3 (or compatible clone)	ATmega328P, USB-B port	1	Pixel Electronics/Nerokas stock Uno-compatible boards
4WD Robot Car Chassis Kit	Acrylic double-deck chassis, 4x DC gear motors (1:48) with wheels, mounting screws/standoffs	1 set	Motors will be wired in left/ right pairs into the two L298N channels
L298N Dual H-Bridge Motor Driver Module	Screw-terminal breakout, onboard 5V regulator with jumper	1	Keep the 5V-enable jumper ON to power the Arduino from the same battery
Battery holder + batteries	6xAA holder with switch and DC leads (7.2–9V), or 2S 18650 pack	1 holder + 6 AA cells	Main power for L298N and (via jumper) the Arduino

USB Cable	Type A–B	1	For uploading code each session
Jumper wire pack	Male-male, male-female, female-female, assorted lengths	1 pack (40+ wires)	Shared classroom stock is fine
IR reflectance sensor module	TCRT5000-based, digital output, onboard potentiometer	2	Front-mounted for line following
HC-SR04 Ultrasonic Sensor	Trig/Echo 4-pin module	1	Front-mounted for obstacle avoidance
SG90 micro servo	9g, 180° hobby servo	1 (extension only)	Only needed for the "scanning sensor" home challenge
Mini breadboard	170-point	1 (optional)	Handy for quick sensor tests before final mounting
Small Philips screwdriver set	#0/#1 tips	1 per 2 teams	For chassis assembly
Zip ties	100mm	10 per team	Cable and sensor management
Hot glue gun and sticks	Low-temp mini glue gun	1 per 4 teams	For securing sensor brackets
Double-sided foam tape	18mm roll	1 small roll per team	Mounting Arduino/L298N to chassis deck
Black insulation tape	20mm width	2 rolls (class set)	Line-follower track construction
White poster board / manila paper	A0-size sheets, 2–3 taped together	1 track per class	Base surface for the black-tape line track (approx. 2m x 1.5m loop)
Obstacle course props	Small cardboard boxes, plastic cups, cones	6–8 per class	For Session 3 obstacle course and Race Day
Laptop with Arduino IDE installed	Windows/Mac, Arduino IDE 1.8+ or 2.x	1 per team	Install and test before Session 1
Stopwatch/timer	Phone app is fine	1 (facilitator)	For Race Day timing
Spare parts kit	1 spare Arduino, 1 spare L298N, spare wheels/motors	Facilitator stock	For fast in-session swaps if a component fails

Session Plans

Session 1: Motorised Chassis

Objectives

- Assemble the 4WD chassis and mount the Arduino and L298N driver securely.
- Correctly wire the L298N to the motors and Arduino, understanding H-bridge direction and PWM speed control.
- Program and test forward, reverse, left-turn, right-turn and stop behaviours on the floor.

Timing (120 minutes)

- 0–10 min: Welcome, safety briefing (battery polarity, pinched fingers from spinning wheels, no shorting bare wires), show a finished car demo, preview the 3-session arc.
- 10–30 min: Chassis assembly — mount all four motors and wheels, attach top/bottom acrylic plates, secure with the supplied screws.
- 30–45 min: Mount the Arduino Uno and L298N module on the top deck using foam tape or zip ties, leaving USB port and screw terminals accessible.
- 45–55 min: Wire the four motors into the L298N — left-side motors in parallel to OUT1/OUT2 (Motor A), right-side motors in parallel to OUT3/OUT4 (Motor B).
- 55–70 min: Wire L298N control pins to the Arduino (ENA, IN1–IN4, ENB) and battery power; facilitator checks every team's wiring before any power-on.
- 70–85 min: Live-code walkthrough — upload the test sketch, verify each wheel spins the correct direction, fix any reversed motors.
- 85–100 min: Upload the full movement sketch; test forward/reverse/turn/stop sequence on the floor.
- 100–115 min: Free-driving challenge — steer the car (by re-uploading code with edited delays) through a simple taped path and a figure-8.
- 115–120 min: Power off, tidy cables, recap the wiring diagram, preview line following.

Step-by-step

1. Open with the safety briefing: batteries must be inserted with correct polarity; hands stay clear of wheels while powered; disconnect the battery before touching any wiring.

2. Hand out chassis kits. Walk the room as teams bolt on motors — check that wheels are on tight and axles spin freely before moving on.
3. Demonstrate mounting the Arduino and L298N on the deck with foam tape, oriented so the USB port faces the back/side for easy cable access during testing.
4. Draw the L298N terminal layout on the board (12V/GND/5V, OUT1–OUT4, ENA/IN1/IN2/IN3/IN4/ENB) and have students wire motors first (car unpowered).
5. Explain H-bridge logic simply: "IN1 HIGH + IN2 LOW spins the motor one way; swap them and it spins the other way; ENA sets how fast." Have students predict directions before testing.
6. Do a full wiring check per team (battery not yet connected) before anyone connects power — check for the common ground between battery, L298N and Arduino.
7. Upload the individual motor test together as a class, line by line, explaining each function.
8. Once every wheel is confirmed spinning the right way, swap in the full movement sketch and let each team run their own tests on the floor.
9. Close with the free-driving challenge and a whole-class debug huddle on any remaining issues.

Build detail

Materials: Arduino Uno, L298N module, 4WD chassis with 4 DC motors, 6xAA battery holder, USB cable, jumper wires, foam tape, zip ties, screwdriver.

Wiring (component → pin):

- Left motor pair (+/–) → L298N OUT1 / OUT2 (Motor A)
- Right motor pair (+/–) → L298N OUT3 / OUT4 (Motor B)
- L298N ENA → Arduino D5 (PWM)
- L298N IN1 → Arduino D8
- L298N IN2 → Arduino D9
- L298N IN3 → Arduino D10
- L298N IN4 → Arduino D11
- L298N ENB → Arduino D6 (PWM)
- L298N 12V terminal → Battery +

- L298N GND terminal → Battery – **and** Arduino GND (common ground is essential)
- L298N 5V OUT → Arduino 5V pin (only with the onboard regulator jumper left in place)

```
// Session 1: Motorised Chassis - Basic Movement Test
// L298N wiring:
// ENA -> D5, IN1 -> D8, IN2 -> D9  (Left motor pair - Motor A)
// IN3 -> D10, IN4 -> D11, ENB -> D6 (Right motor pair - Motor B)

const int ENA = 5;
const int IN1 = 8;
const int IN2 = 9;
const int IN3 = 10;
const int IN4 = 11;
const int ENB = 6;

const int SPEED = 180; // 0-255, start moderate

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
  Serial.begin(9600);
  Serial.println("Motorised Chassis Test Ready");
}

void forward(int speed) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void backward(int speed) {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void turnLeft(int speed) {
  // left wheels reverse, right wheels forward -> pivot left
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}
```

```
void turnRight(int speed) {
  // left wheels forward, right wheels reverse -> pivot right
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed);
  analogWrite(ENB, speed);
}

void stopCar() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void loop() {
  Serial.println("Forward");
  forward(SPEED);
  delay(1500);

  Serial.println("Stop");
  stopCar();
  delay(500);

  Serial.println("Backward");
  backward(SPEED);
  delay(1500);

  Serial.println("Stop");
  stopCar();
  delay(500);

  Serial.println("Turn Left");
  turnLeft(SPEED);
  delay(700);

  Serial.println("Stop");
  stopCar();
  delay(500);

  Serial.println("Turn Right");
  turnRight(SPEED);
  delay(700);

  Serial.println("Stop");
  stopCar();
  delay(2000);
}
```

Checks for understanding

- What happens to a motor's direction if you swap its IN1 and IN2 wires? Why?
- Why must the battery, L298N and Arduino all share a common ground?

- What does the number we pass into `analogWrite(ENA, speed)` actually control?

Common mistakes & fixes

MISTAKE	FIX
One wheel spins the wrong way	Swap that motor's two wires at the OUT terminal (don't change the code)
Car doesn't move at all	Check the common ground link between battery, L298N GND and Arduino GND
Arduino resets/browns out when motors start	Batteries are weak or the 5V jumper path is overloaded — use fresh batteries or power the Arduino separately via USB during testing
L298N module feels hot to the touch	Check for a short in motor wiring or stalled wheels; power off immediately and inspect

Session 2: Line Follower

Objectives

- Understand how IR reflectance sensors distinguish black line from white surface.
- Mount, wire and calibrate two IR sensors on the front of the chassis.
- Program a closed-loop line-following algorithm and test it on a real track.

Timing (120 minutes)

- 0–10 min: Recap Session 1 wiring/code; quick demo drive.
- 10–20 min: Explain IR reflectance sensing, digital output, and the onboard calibration potentiometer.
- 20–35 min: Mount two IR sensors on the front of the chassis (~1cm above the ground), wire to Arduino.
- 35–50 min: Build/verify the class line track; calibrate each sensor's potentiometer over black and white sections using the onboard indicator LED.
- 50–65 min: Upload a sensor-test sketch, read raw values on Serial Monitor, confirm HIGH/LOW logic matches black/white.
- 65–90 min: Build the full line-following sketch together, explaining the decision logic step by step.
- 90–110 min: Test runs on the track; debug oscillation, missed turns, sensor spacing.
- 110–120 min: Group reflection on what worked; preview obstacle avoidance.

Step-by-step

1. Recap the H-bridge functions from Session 1 (they'll be reused unchanged).
2. Explain IR sensing: infrared LED shines down, black absorbs it, white reflects it back to the photodiode; the module's comparator turns that into a clean HIGH/LOW signal.
3. Help each team mount sensors at the very front, side by side, roughly 3–4cm apart, close enough to the floor to get a strong reading but not touching.
4. Lay out (or reveal, if pre-built) the class track: a closed loop of black tape on white poster board, including at least one curve and one straight.
5. Run the sensor-test sketch as a class; have each team slide their car over black and white sections and read the Serial Monitor values, then tune the potentiometer until the reading flips reliably at the tape edge.
6. Introduce the full line-following sketch on the board, walking through each `if/else` branch and what each sensor combination physically means.
7. Let teams upload and test on the shared track, cycling through in small groups; circulate to help with oscillation (usually fixed by increasing sensor spacing or lowering turn speed) and missed sharp turns.
8. Close with a short discussion: what would happen if both sensors were mounted too close together, or too far apart?

Build detail

Materials: 2x IR reflectance sensor modules, jumper wires, zip ties/hot glue for mounting, black-tape track on white poster board.

Wiring (component → pin):

- Left IR sensor VCC → Arduino 5V, GND → Arduino GND, OUT → Arduino D2
- Right IR sensor VCC → Arduino 5V, GND → Arduino GND, OUT → Arduino D3
- Motor wiring unchanged from Session 1

```
// Session 2a: Sensor calibration test - run this first
// Left sensor OUT -> D2, Right sensor OUT -> D3

void setup() {
  pinMode(2, INPUT);
  pinMode(3, INPUT);
  Serial.begin(9600);
}

void loop() {
```

```
  int leftValue = digitalRead(2);
  int rightValue = digitalRead(3);
  Serial.print("Left: ");
  Serial.print(leftValue);
  Serial.print("   Right: ");
  Serial.println(rightValue);
  delay(200);
  // Slide the car over black tape and white board while watching these values.
  // Adjust each sensor's potentiometer until black reliably reads LOW and white reads HIGH.
}
```

```
// Session 2b: Line Follower - full autonomous run
// IR sensors: Left -> D2, Right -> D3 (LOW = black line, HIGH = white surface)
// Motor wiring identical to Session 1

const int ENA = 5;
const int IN1 = 8;
const int IN2 = 9;
const int IN3 = 10;
const int IN4 = 11;
const int ENB = 6;

const int leftSensor = 2;
const int rightSensor = 3;

const int SPEED = 150;
const int TURN_SPEED = 170;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(leftSensor, INPUT);
  pinMode(rightSensor, INPUT);
  Serial.begin(9600);
}

void forward(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void turnLeft(int speed) {
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void turnRight(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void stopCar() {
```

```
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

void loop() {
  int leftValue = digitalRead(leftSensor);
  int rightValue = digitalRead(rightSensor);

  if (leftValue == HIGH && rightValue == HIGH) {
    // both sensors on white -> line is centred -> go straight
    forward(SPEED);
  }
  else if (leftValue == LOW && rightValue == HIGH) {
    // left sensor found black -> line drifting left -> steer left
    turnLeft(TURN_SPEED);
  }
  else if (leftValue == HIGH && rightValue == LOW) {
    // right sensor found black -> line drifting right -> steer right
    turnRight(TURN_SPEED);
  }
  else {
    // both sensors on black -> junction or stop marker -> pause briefly
    stopCar();
    delay(200);
  }
}
```

Checks for understanding

- Why does the sensor read LOW over black tape and HIGH over the white board?
- What would you change in the code if your sensor module's logic were flipped (HIGH on black)?
- The car keeps zig-zagging wildly along straights — what two things could you try to fix it?

Common mistakes & fixes

MISTAKE	FIX
Sensor mounted too high above the track	Lower it to ~1cm; reflectance sensors are unreliable beyond a few cm
Sensors always read the same value regardless of surface	Recalibrate the onboard potentiometer while watching Serial Monitor
Car can't handle sharp curves	Move sensors slightly closer together, or slow down turn speed
Car overshoots and loses the line entirely	Reduce SPEED/TURN_SPEED values, or narrow the black tape width consistently

Session 3: Obstacle Avoidance & Race Day

Objectives

- Understand how the HC-SR04 measures distance using an ultrasonic pulse and echo timing.
- Mount, wire and code obstacle detection so the car reacts to objects in its path.
- Finish integrating all three builds into a complete autonomous car and compete in Race Day.

Timing (120 minutes)

- 0–15 min: Recap; explain HC-SR04 principle (speed of sound, trigger pulse, echo pulse width); demo distance readout on Serial Monitor.
- 15–30 min: Mount the HC-SR04 at the front of the chassis; wire to Arduino.
- 30–50 min: Upload and test a distance-reading sketch; calibrate a safe stopping threshold (e.g. 15cm).
- 50–75 min: Build the full obstacle-avoidance sketch (drive, detect, stop, reverse, turn, resume); test and tune.
- 75–95 min: Run cars through a live obstacle course (boxes/cones); iterate threshold and turn timing.
- 95–105 min: Optional — advanced teams upload the combined line + obstacle "bonus" sketch for a full-autonomy demo.
- 105–115 min: Race Day heats — timed runs through the track/obstacle course; celebrate results.
- 115–120 min: Awards, pack-down of shared class materials, take-home checklist.

Step-by-step

1. Explain the HC-SR04: it sends a 10-microsecond trigger pulse, the sensor emits an ultrasonic burst, and ECHO goes HIGH for exactly as long as it takes the sound to bounce back — distance = (time × speed of sound) ÷ 2.
2. Mount the sensor at the very front, level and facing forward, using a bracket, zip ties or hot glue.
3. Wire TRIG and ECHO to the Arduino; run the distance-test sketch as a class and have students hold a hand at varying distances to watch the Serial Monitor values change.

4. Introduce the `readDistanceCM()` function line by line, especially the `pulseIn()` call and the timeout parameter (explain why a timeout prevents the code from freezing if no echo returns).
5. Build the obstacle-avoidance sketch together: drive forward while clear, and when an object is closer than the safe distance, stop, reverse briefly, turn, and resume.
6. Set up a simple obstacle course with boxes/cones and let teams test and tune their SAFE_DISTANCE and turn timing.
7. Offer the combined "bonus" sketch to any team ready for it — it merges Session 2's line sensors with Session 3's ultrasonic sensor into one autonomous loop.
8. Run Race Day: each car gets a timed heat on the shared track/obstacle course; track completion time and any obstacle collisions as scoring criteria.
9. Wrap up with awards (fastest, most reliable, best comeback, etc.) and a take-home materials checklist so nothing is left behind.

Build detail

Materials: HC-SR04 sensor, mounting bracket/zip ties/hot glue, jumper wires, obstacle course props.

Wiring (component → pin):

- HC-SR04 VCC → Arduino 5V
- HC-SR04 GND → Arduino GND
- HC-SR04 TRIG → Arduino D7
- HC-SR04 ECHO → Arduino D4
- Motor wiring unchanged from Session 1; IR sensors unchanged from Session 2 (kept in place for the bonus combined sketch)

```
// Session 3a: Obstacle Avoidance
// HC-SR04: TRIG -> D7, ECHO -> D4
// Motor wiring identical to Session 1

const int ENA = 5;
const int IN1 = 8;
const int IN2 = 9;
const int IN3 = 10;
const int IN4 = 11;
const int ENB = 6;

const int trigPin = 7;
const int echoPin = 4;
```

```
const int SPEED = 160;
const int SAFE_DISTANCE = 15; // cm - obstacle threshold

void setup() {
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void forward(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void backward(int speed) {
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void turnRight(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void stopCar() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}

long readDistanceCM() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000); // 30ms timeout, ~5m max range
  if (duration == 0) return 999; // no echo received - treat as clear path
  long distance = duration * 0.0343 / 2; // speed of sound 343 m/s, round trip halved
  return distance;
}

void loop() {
  long distance = readDistanceCM();
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  if (distance > SAFE_DISTANCE) {
    forward(SPEED);
  } else {
    stopCar();
    delay(200);
    backward(SPEED);
    delay(400);
```

```
    stopCar();
    delay(200);
    turnRight(SPEED);
    delay(500);
    stopCar();
    delay(200);
  }
}
```

```
// Session 3b (Bonus): Full autonomous mode
// Combines Session 2 line sensors + Session 3 ultrasonic sensor.
// Follows the black line by default; if an obstacle blocks the line,
// stops, reverses, steers around, then resumes line-following.

const int ENA = 5, IN1 = 8, IN2 = 9, IN3 = 10, IN4 = 11, ENB = 6;
const int leftSensor = 2, rightSensor = 3;
const int trigPin = 7, echoPin = 4;

const int SPEED = 150;
const int TURN_SPEED = 170;
const int SAFE_DISTANCE = 15;

void setup() {
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(leftSensor, INPUT); pinMode(rightSensor, INPUT);
  pinMode(trigPin, OUTPUT); pinMode(echoPin, INPUT);
  Serial.begin(9600);
}

void forward(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void backward(int speed) {
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void turnLeft(int speed) {
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void turnRight(int speed) {
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  analogWrite(ENA, speed); analogWrite(ENB, speed);
}

void stopCar() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

```
long readDistanceCM() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  if (duration == 0) return 999;
  return duration * 0.0343 / 2;
}

void loop() {
  long distance = readDistanceCM();

  if (distance <= SAFE_DISTANCE) {
    // obstacle blocking the track - avoid it
    stopCar(); delay(150);
    backward(SPEED); delay(300);
    stopCar(); delay(150);
    turnRight(SPEED); delay(500);
    stopCar(); delay(150);
    return; // re-check sensors fresh next loop
  }

  int leftValue = digitalRead(leftSensor);
  int rightValue = digitalRead(rightSensor);

  if (leftValue == HIGH && rightValue == HIGH) {
    forward(SPEED);
  } else if (leftValue == LOW && rightValue == HIGH) {
    turnLeft(TURN_SPEED);
  } else if (leftValue == HIGH && rightValue == LOW) {
    turnRight(TURN_SPEED);
  } else {
    stopCar();
    delay(150);
  }
}
```

Checks for understanding

- Why do we divide by 2 in the distance calculation?
- What does the 30000-microsecond timeout in `pulseIn()` protect against?
- In the combined sketch, why does the obstacle-avoidance check run before the line-following check in the loop?

Common mistakes & fixes

MISTAKE	FIX
Distance readings are wildly inconsistent or always 999	Check TRIG/ECHO aren't swapped; ensure sensor faces forward with nothing blocking it at close range
Car detects "obstacles" that aren't there	

	Sensor picking up the floor or its own mounting bracket — angle it slightly upward or reposition
Car always turns into a wall on one side of the course	SAFE_DISTANCE too small or turn duration too short — increase both slightly
Combined sketch drives fine but never leaves obstacle-avoid mode	Check the SAFE_DISTANCE threshold isn't triggered by the track surface itself; raise the sensor mount slightly

Assessment & Showcase

Assessment is formative and hands-on across all three sessions rather than written tests. At the end of each session, the facilitator checks off three things per student/team: (1) hardware assembled and wired correctly, (2) that session's sketch uploads and runs without errors, and (3) the car demonstrates the target behaviour live (moves on command in Session 1, follows the track in Session 2, avoids an obstacle in Session 3). A simple tick-sheet per team across the three sessions is enough to track progress and flag anyone who needs extra support before Race Day.

Race Day is the summative showcase. Every car runs the same standardised course: a taped line loop with at least one obstacle placed directly on the track. Each team gets two timed heats; the better time counts. Scoring combines completion time with a small penalty added per obstacle collision or per time the car needs a manual restart, so both speed and reliability matter. "Done" for the course means: the car reliably drives forward/reverse/turns on command, follows the black-tape track for at least one full lap without needing to be picked up, and stops/manoeuvres around a placed obstacle without a collision, in either the standalone obstacle-avoidance sketch or the bonus combined sketch. Announce category awards (Fastest Lap, Most Reliable, Best Recovery, Most Creative Chassis Decoration) so every team leaves with a win to celebrate, and finish by taking a group photo with all the cars lined up on the track.

Extension & Home Challenges

- **Easy:** Add an LED "headlight" that lights up automatically whenever the car is moving forward, and a second LED that flashes whenever the ultrasonic sensor detects an obstacle within range.

- **Medium:** Add a potentiometer wired to an analog pin so the driver can dial the car's base speed up or down live, and modify the code to read it each loop instead of using a fixed SPEED constant.
- **Hard:** Mount the HC-SR04 on the SG90 servo so it can sweep left-right at each obstacle stop, measure distance at three angles (left/centre/right), and steer toward whichever side has more open space — then merge this smarter avoidance logic with the Session 2 line-follower so the car can leave the line to dodge an obstacle and correctly find its way back onto the track afterward.